

"With What Rationale" as a Standalone Accountability Question: Conditional Rationale Capture in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 4, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the fourth of the four accountability questions named in the source paper's path-retraceability vocabulary — *with what rationale* — as a standalone architectural commitment with independent operational content, separable from the other three questions with which it composes, and distinguished from them by a conditional structure that is itself part of the architectural commitment.

Abstract

The CKS pattern names four accountability questions every piece of substrate content produced by deployment activity must support: *what was decided*, *by whom*, *under what authority*, and *with what rationale*. Companion notes formalize the first three as standalone commitments under the integrating frame the path-retraceability decomposition establishes. This note formalizes the fourth — “with what rationale” — and identifies the structural feature that distinguishes it from the other three: rationale capture is conditional, not universal. Rationale is required as substrate content when the orchestration rule that authorized the write or the substrate schema for the content specifies that rationale must be captured; otherwise it is permitted but not architecturally required. The conditional structure is not a softening of the commitment; it *is* the commitment. The note defines four operational components that together constitute the architectural commitment, distinguishes the commitment from four adjacent patterns commonly conflated with it (mandatory rationale on every write, free-form comments, post-hoc rationale generation, rationale held in external systems), enumerates nine failure modes, and provides an operational test for whether a system's rationale capture is CKS-coherent specifically.

1. Why “with what rationale” needs to be formalized as standalone

The path-retraceability commitment in §3.1 of the source paper imports the accountability vocabulary from Naja, Markovic, Edwards, and Cottrill (2021) and binds it to a four-question structure: a substrate is path-retraceable when readers can recover *what was decided*, *by whom*, *under what authority*, and *with what rationale* from substrate content alone. The integrating-frame note for the path-retraceability decomposition establishes these four questions as a structured set; companion notes formalize “what was decided,” “by whom,” and “under what authority” as standalone commitments. This note formalizes the fourth.

The motivating observation is that rationale capture in deployments drifts. The same workflow can record rationale richly for some decisions, sparsely for others, in substrate for some writes, in external decision-logging tools for others, and in the writer’s memory for the rest. Without a precise architectural specification, deployments cannot reason about which rationale-capture obligations the architecture itself imposes, and downstream implementers cannot defensibly claim CKS-coherence on this axis. The first contribution of this note is the precise specification.

The second contribution is structural. The other three accountability questions are universal — every piece of substrate content produced by deployment activity must carry them. “With what rationale” is conditional — it is required when the rule that authorized the write or the schema for the content specifies it, and permitted but not architecturally required otherwise. The conditional structure forecloses two adjacent architectures simultaneously: those that treat rationale as always-required overhead applied to every decision, and those that treat rationale as always-optional convenience the architecture neither enforces nor requires. The CKS commitment is neither; it is the third option, in which the requirement is architecturally enforced where the deployment’s rules and schema specify, and architecturally permitted (but not enforced) elsewhere.

A third motivation is the relationship to the conflict-specific provenance treatment in the conflict-as-first-class decomposition. That treatment names rationale as one of four conflict-specific provenance fields with the conditional qualifier noted in passing. This note is the broader treatment — rationale across all substrate writes, conflicts and non-conflicts alike — and the two share content on the conditional structure but differ in scope.

2. The “with what rationale” commitment, defined precisely

The architectural commitment to “with what rationale” has four operational components.

(a) Rationale capture is conditional. Every piece of substrate content produced by deployment activity may or may not be architecturally required to carry rationale; the requirement depends on whether the orchestration rule that authorized the write or the substrate schema for the content specifies that rationale must be captured. Implementations that treat rationale as universally required for every write, or as universally optional regardless of rule or schema, fail to capture the conditional structure architecturally. The condition itself is set at design time through rule authoring and schema design, not at write time by individual writers.

(b) Where required, rationale is substrate content. When the rule or schema specifies rationale capture, the rationale is recorded as substrate metadata for the content, readable through normal substrate inspection per the inspect-right requirements that ground the human-governed commitment. The rationale itself is not held in external systems, comment threads, or operational logs; it lives in substrate where substrate-only paths can resolve it.

(c) Where required, rationale is committed atomically with the content. The cell-to-substrate write that produces the substrate content commits the content and its rationale as a single architectural commit. There is no window during which rationale-required substrate content exists without rationale, no deferral to a later capture step, no eventual-consistency relationship between content and rationale. The commit is one event.

(d) Where required, rationale is sufficient for retraceability. The rationale captures the writer’s reasoning for the decision in substantive form — not placeholder text, not empty fields, not opaque codes, not pointers to external systems. A reader exercising the inspect right can examine the rationale and recover why the decision was made. What counts as sufficient depends on the rule’s or schema’s specification; the architectural commitment is to substantive content where required, not to a particular length, format, or rhetorical register.

The four components together define the architectural commitment. A system that satisfies fewer than four cannot reliably answer “with what rationale” for substrate content where the rationale is required.

3. What the commitment does NOT claim

The standalone treatment is not a maximalist treatment. Stating precisely what the commitment does not claim is what keeps the standalone framing from drifting into something stronger than the source paper supports.

It does not claim rationale is required for every write. The conditional structure is what distinguishes “with what rationale” from the other three accountability questions. The architecture supports both rationale-rich deployments (rules and schema specify rationale capture broadly) and rationale-light deployments (rules and schema specify rationale capture narrowly, perhaps only for critical decisions or exceptional resolutions). Both are CKS-coherent.

It does not specify a minimum length or format for rationale. Rationale may be a brief note, a structured explanation, or a multi-paragraph essay. Format is determined by the rule’s or schema’s specification, not by the architecture. A single-sentence rationale that captures the writer’s reasoning substantively is as architecturally valid as a multi-paragraph one.

It does not require rationale to be human-authored. For LLM-mediated cell writes under the AI-as-substrate-mediator commitment, the LLM may produce the rationale string as part of its write, attributed to the LLM under the substrate-mediator’s writes-recorded-with-attribution property. The architecture commits to rationale being captured in substrate where rules require it, not to who or what produced the rationale. The LLM does not exercise authority by producing rationale; it produces rationale within the rule’s specification.

It does not require rationale to be approved before capture. The commitment is to capture at write time per the atomic-with-content commit; review, approval, refinement, or annotation of rationale after capture is a deployment concern, not an architectural one. A deployment may layer post-write review processes over substrate without altering the architectural commitment.

It does not specify how rationale interacts with conflict resolution. The conflict-specific provenance treatment in the conflict-as-first-class decomposition addresses rationale as one of four conflict-specific provenance fields; the broader treatment in this note covers rationale across all writes. For conflict-related writes, both apply; the relationship is operational rather than architecturally distinctive.

It does not require rationale to be true, accurate, or well-reasoned. Rationale captures the writer’s reasoning as the writer expressed it; whether that reasoning is sound is the reader’s evaluation, not the architecture’s. The architectural commitment is to the rationale being captured, not to its quality.

4. What the commitment is NOT

Four adjacent patterns are commonly conflated with the “with what rationale” commitment. Each is a real and reasonable pattern in some other architecture; naming what the commitment is not is what prevents the misreadings.

Not mandatory rationale on every write. Some implementations treat rationale as universally required for all substrate writes, with rationale-capture overhead applied to every decision regardless of content type or rule. The CKS architectural commitment is conditional; mandatory-everywhere rationale fails because it removes the deployment’s ability to configure rationale requirements through rules and schema. Mandatory-everywhere is also not “more careful CKS” — it is a different architecture with a different cost profile.

Not free-form comments. Some implementations capture rationale-like content as free-form comments attached to substrate content. Comments can be useful but fail the architectural commitment if they are not the rationale field — that is, if a reader cannot identify which comment is the rationale per the substrate’s schema. The commitment is to rationale as a structured part of substrate provenance, queryable as such, not to free-form annotations a reader must interpret.

Not post-hoc rationale generation. Some implementations generate rationale after the write through inference (analyzing decision content, asking the writer to provide rationale after the fact, generating LLM explanations of past decisions). Post-hoc generation fails the architectural commitment because it is not committed atomically with the content; the substrate has a window during which rationale-required content lacks rationale, and the rationale eventually captured may be reconstruction rather than the writer’s reasoning at the time of the write.

Not rationale held in external systems. Some implementations capture rationale in external systems — decision-logging applications, governance dashboards, chat threads, ticketing tools — while leaving the substrate’s rationale field empty or carrying a pointer to the external record. The architectural commitment is to rationale being substrate content, queryable through substrate-only paths. External-only rationale fails the commitment because the substrate cannot answer “with what rationale” from substrate alone; resolution requires traversal into systems the substrate-as-source-of-truth commitment treats as non-authoritative for coordination state.

5. Why “with what rationale” is load-bearing for downstream commitments

The standalone commitment underwrites several CKS commitments that depend on it.

Conflict-as-first-class. Cell-level conflict resolution decisions often require rationale per the rule that governs resolution; the rationale is what makes resolution auditable and is part of the resolution’s substrate footprint. Without the conditional rationale-capture commitment, resolution would be observable only as the selection outcome (“X was chosen over Y”) with the reasoning behind the selection unrecoverable from substrate, and resolution decisions would lack the architectural slot that distinguishes well-considered resolutions from arbitrary ones.

The path-retraceability integrating frame. Paths through substrate include rationale where required; readers traversing paths see not just decisions and authority but the reasoning. Without “with what rationale” as

substrate content, paths would have rationale-shaped gaps at decisions where the rule specified reasoning capture, and the integrating frame would no longer hold over those gaps.

Substrate-only paths. Paths must be reconstructible from substrate alone. Rationale held externally — in decision logs, governance dashboards, chat history — would break this property at every decision where rationale is required and the substrate’s rationale field is empty or external-pointing.

The labor allocation framework. Direct human labor often produces rationale-rich writes because humans naturally explain reasoning when making decisions; LLM labor under orchestration rules produces rationale when rules specify it. The conditional rationale-capture commitment is what makes the framework operationally complete: human and LLM-mediated decisions both carry rationale where rules require, with differences in how rationale is produced but consistent architectural treatment of the captured rationale.

6. Failure modes that violate the commitment

Nine anti-patterns name ways an implementation can fail the architectural commitment.

(a) Universal-mandatory rationale. The implementation requires rationale on every write, removing the deployment’s ability to configure rationale requirements through rules and schema. The conditional structure is broken in the maximalist direction.

(b) Universal-optional rationale. The implementation treats rationale as never architecturally required; rules and schema specifications for rationale capture are not enforced. The conditional structure is broken in the minimalist direction.

(c) Free-form comment substitution. Rationale-required content has empty or absent rationale fields while rationale-like content lives in free-form comments, chat threads, or annotation surfaces. Readers cannot identify which content is the rationale per substrate schema, and substrate-only paths cannot resolve it.

(d) Post-hoc rationale capture. Rationale is captured after the write rather than atomically with the content. The substrate has a window during which rationale-required content lacks rationale, and the rationale eventually captured may not reflect the writer’s reasoning at the time of the write.

(e) External-system rationale. Rationale lives in external decision-logging tools, governance dashboards, ticketing systems, or chat threads. The substrate’s rationale field carries placeholder text, an external reference, or nothing. Substrate-only paths are broken at every required-rationale write.

(f) Placeholder rationale. The rationale field is populated with non-substantive content — “see attached,” “as discussed,” “per process,” “TBD,” default-string boilerplate, or opaque codes — that does not capture the writer’s reasoning. The capture commitment is satisfied formally but defeated in substance.

(g) Selective rationale enforcement. The implementation enforces rationale capture for some content types where rules or schema specify it but ignores rationale requirements for other content types. The architectural enforcement is partial and unpredictable to readers.

(h) Rationale stripped from migrated substrate. Substrate migration between hosts strips, abbreviates, or transforms rationale in ways that lose substantive content. The migrated substrate has rationale fields but they no longer support retraceability — a particularly insidious failure because formal compliance is preserved while the commitment is hollowed out.

(i) Rationale degraded over time. Automated processes — compression, normalization, schema migration, summarization passes, archival reformatting — strip, abbreviate, or otherwise degrade rationale content over the substrate’s lifetime. The substrate’s commitment to rationale being architecturally captured is eroded gradually rather than violated at any single moment.

7. Operational test

A system satisfies the “with what rationale” commitment if and only if all of the following are true at all times during the substrate’s existence.

1. For substrate content where the orchestration rule that authorized the write or the substrate schema for the content specifies rationale capture, the substrate contains substantive rationale as substrate metadata for the content.
2. For substrate content where rationale is not required by rule or schema, the substrate may or may not contain rationale; the architecture supports both patterns without architectural enforcement either way.
3. Where required, rationale is committed atomically with the content at the cell-to-substrate write — not in a deferred step, not in an eventually-consistent relationship, not subject to a later capture process.
4. Where required, rationale is substantive — not placeholder text, not opaque codes, not external references, not boilerplate strings the schema accepts but readers cannot use.
5. Rationale, where required, is queryable from substrate alone through standard read operations; answering “with what rationale” does not require traversal into external decision-logging systems, governance dashboards, comment surfaces, or operational logs.

A system that fails any of (1)–(5) does not satisfy the “with what rationale” commitment in the architectural sense, even if it provides rationale tracking in some other form, in some other layer, or by some other mechanism.

8. Conclusion

Implementations under pressure to streamline decision-making, reduce capture overhead, or simplify governance consistently drift toward rationale patterns that fail one or more architectural components. The drift is steady because rationale capture feels like overhead, external decision-logging tools are operationally familiar, and post-hoc generation appears to recover rationale at lower friction than atomic-with-content capture imposes.

The drift’s downstream consequences manifest where the commitment is most load-bearing. Conflict-resolution opacity surfaces when rationale required by the resolution rule is missing or external. Governance failures surface when humans exercising the inspect right cannot recover the reasoning behind decisions made under their authority. Substrate-only-paths failures surface when path traversal hits required-rationale writes whose rationale resolution requires external systems. Labor-allocation framework gaps surface when LLM-mediated decisions lack the reasoning capture the rules specify, and the architectural distinction between rule-authorized LLM writes and ungoverned LLM outputs erodes.

Naming “with what rationale” as a standalone architectural commitment — with the four components in §2, the limitations in §3, the four adjacent-pattern distinctions in §4, the load-bearing connections in §5, the nine

failure modes in §6, and the operational test in §7 — gives downstream implementers a precise specification of what the commitment to conditional rationale capture requires. With this note in place alongside its companions for “what was decided,” “by whom,” and “under what authority,” the four accountability questions are formalized as standalone commitments under the path-retraceability integrating frame.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). “*With What Rationale*” as a Standalone Accountability Question: *Conditional Rationale Capture in the Coordination Knowledge Substrate Pattern*. May 4, 2026. ORCID: 0009-0004-8065-3235.